



10주차 발표

4조: 김서영 박린 안지현

목차

#5.0~5.2 토큰화 개요 ~ 형태소 토큰화

#5.3 하위 단어 토큰화

#6 임베딩



5.0 토큰화 개요



5.0 토큰화 개요

자연어와 자연어 처리

- 자연어(자연 언어): 사람들이 쓰는 언어 활동을 위해 자연히 만들어진 언어
- 자연어 처리(NLP): 컴퓨터가 인간의 언어를 이해하고 해석 및 생성하기 위한 기술
 - 인공지능의 하위 분야, 컴퓨터가 인간과 유사한 방식으로 인간의 언어를 이해하고 처리하는 것이 주요 목표
 - 모델 개발을 위해 모호성, 가변성, 구조의 문제가 해결되어야 함

5.0 토큰화 개요

토큰화

- 자연어 처리 모델을 만들기 위해 **말뭉치**를 **토큰**으로 나누어야 함

* 말뭉치: 자연어 모델을 훈련하고 평가하는 데 사용되는 대규모의 자연어

* 토큰: 개별 단어나 문장 부호와 같은 텍스트

* 토큰화: 컴퓨터가 자연어를 이해할 수 있도록 토큰으로 나누는 과정, 토큰나이저(텍스트 문자열을 토큰으로 나누는 알고리즘/소프트웨어)를 사용

- 토큰나이저를 사용한 문장의 토큰화 예시

■ 입력: '형태소 분석기를 이용해 간단하게 토큰화할 수 있다.'

■ 결과: ['형태소', '분석기', '를', '이용', '하', '어', '간단', '하', '게', '토큰', '화', '하', 'ㄴ', '수', '있', '다', '.']

- 토큰나이저를 구축하는 다양한 방법이 존재함

- 공백 분할

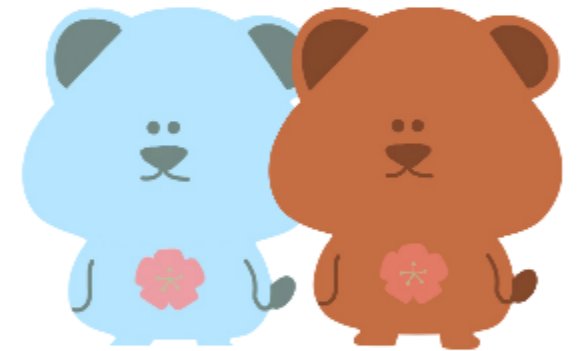
- 정규표현식 적용

- 어휘 사전 적용(사전에 정의된 단어를 활용, 따라서 없는 단어나 토큰(OOV)이 존재할 수 있음, 이를 고려해 토큰화해야 함)

- 머신러닝 활용

=> 토큰나이저 다양한 구축 방법은 시스템의 성능/처리 결과에 큰 영향을 주기도 함!

5.1 단어 및 글자 토큰화



5.1 단어 및 글자 토큰화

단어 토큰화

- 텍스트 데이터를 의미 있는 단위인 **단어로 분리**하는 작업
- 자연어 처리 분야에서 핵심적인 전처리 작업 중 하나
- 품사 태깅, 개체명 인식, 기계 번역 등의 작업에서 널리 사용됨
- 가장 일반적인 토큰화 방법

```
review = " 현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"  
tokenized = review.split()  
print(tokenized)
```

```
['현실과', '구분', '불가능한', 'cg.', '시각적', '즐거움은', '최고!', '더불어', 'ost는', '더더욱', '최고!!']
```

- Split 메서드를 사용해 주어진 구분자를 통해 문자열을 나눔(구분자 입력하지 않으면 공백을 기준으로)

5.1 단어 및 글자 토큰화

글자 토큰화

- 글자 단위로 문장을 나누는 방식
- 비교적 작은 단어 사전을 구축할 수 있다는 장점
 - > 컴퓨터 자원의 효율적 사용 가능, 각 단어를 더 자주 학습할 수 있음
- 언어 모델링과 같은 시퀀스 예측 작업에서 활용

```
review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"  
tokenized = list(review)  
print(tokenized)
```

```
['현', '실', '과', ' ', '구', '분', ' ', '불', '가', '능', '한', ' ', 'c', 'g', '.', ' ', '시', '각', '적', ' ', '즐', '거', '움', '은', ' ', '최', '고', '!', ' ', '더', '불어', ' ', 'o', 's', 't', '는', ' ', '더', '더', '욱', ' ', '최', '고', '!!']
```

5.1 단어 및 글자 토큰화

```
) from jamo import h2j, j2hcj

review = "현실과 구분 불가능한 cg. 시각적 즐거움은 최고! 더불어 ost는 더더욱 최고!!"
decomposed = j2hcj(h2j(review))
tokenized = list(decomposed)
print(tokenized)

['ㅎ', 'ㄷ', 'ㄴ', 'ㅅ', 'ㅣ', 'ㄹ', 'ㄱ', 'ㅍ', ' ', 'ㄱ', 'ㅈ', 'ㅂ', 'ㅈ', 'ㄴ', ' ', 'ㅂ', 'ㅈ', 'ㄹ', 'ㄱ', 'ㅅ', 'ㄴ', ' ']
```

- 자모(jamo) 라이브러리를 활용한 토큰화
 - ➔ 비교적 적은 크기의 단어 사적 구축 가능
 - ➔ 단어 토큰화의 단점이었던 접사와 문장 부호의 의미 학습 가능
 - ➔ OOV 획기적으로 줄일 수 있음
- 하지만 개별 토큰의 의미가 없으므로 자연어 모델이 각 토큰의 의미를 조합해 결과를 도출해야 함
 - ➔ 이에 따른 단점이 존재
 - 다의어나 동음이의어가 많은 도메인에서의 구별 어려움
 - 모델 입력 시퀀스의 길이가 길어질 수록 연산량 증가

5.2 형태소 토큰화



5.2 형태소 토큰화

형태소 토큰화

- 텍스트를 형태소 단위로 나누는 토큰화 방법
- 언어의 문법과 구조를 고려해 단어를 분리하고 이를 의미 있는 단위로 분류
- 교착어인 언어에서 중요하게 수행됨 (한국어가 여기에 해당)

용어 정리

- 형태소: 실제로 의미를 가지고 있는 최소의 단위
 - 자립 형태소: 스스로 의미를 가지고 있는 형태소, 단어를 이루는 기본 단위
 - 의존 형태소: 스스로 의미를 갖지 못하고 다른 형태소와 조합되어 사용되는 형태소

5.2.1 형태소 어휘 사전

형태소 어휘 사전

- 자연어 처리에서 사용되는 단어의 집합인 어휘 사전 중, 각 단어의 형태소 정보를 포함하는 사전
- 단어가 어떤 형태소들의 조합으로 이루어져 있는지에 대한 정보를 담고 있음
- 각 형태소가 어떤 품사에 속하는지, 해당 품사의 뜻 등의 정보를 함께 제공
- 품사 태깅: 텍스트 데이터를 형태소 분석하여 해당하는 품사를 태깅하는 작업
→ 이를 통해 자연어 처리 분야에서 문맥을 고려할 수 있음

5.2.1 형태소 토큰화 라이브러리_KoNLPy

KoNLPy

- **한국어 자연어 처리**를 위해 개발된 라이브러리
- 명사 추출, 형태소 분석, 품사 태깅 등의 기능 제공
- Okt, 꼬꼬마(Kkma), 코모란(Komoran) 등의 다양한 형태소 분석기 지원

Okt

- 문장을 입력 받아 명사, 구, 형태소, 품사 등의 정보 추출하는 메서드 제공
- 대표적인 메서드
 - 명사 추출(okt.nouns)
 - 구문 추출(okt.phrases)
 - 형태소 추출(okt.morphs)
 - 품사 태깅(okt.pos)

```
from konlpy.tag import Okt

okt = Okt()

sentence = "무엇이든 상상할 수 있는 사람은 무엇이든 만들어 낼 수 있다."

nouns = okt.nouns(sentence)
phrases = okt.phrases(sentence)
morphs = okt.morphs(sentence)
pos = okt.pos(sentence)

print("명사 추출 :", nouns)
print("구 추출 :", phrases)
print("형태소 추출 :", morphs)
print("품사 태깅 :", pos)
```

5.2.1 형태소 토큰화 라이브러리_KoNLPy

꼬꼬마(Kkma)

- 명사, 문장, 형태소, 품사 추출
- 구문 추출 기능을 제공하지 않음!

```
from konlpy.tag import Kkma

kkma = Kkma()

sentence = "무엇이든 상상할 수 있는 사람은 무엇이든 만들어 낼 수 있다."

nouns = kkma.nouns(sentence)
sentences = kkma.sentences(sentence)
morphs = kkma.morphs(sentence)
pos = kkma.pos(sentence)

print("명사 추출 :", nouns)
print("문장 추출 :", sentences)
print("형태소 추출 :", morphs)
print("품사 태깅 :", pos)
```

KoNLPy의 라이브러리

- 동일한 메서드라도 분석기에 따라 다른 결과가 나타남
 - 품사 태깅 역시 상이한 결과 제공
- ➔ 시스템의 목적과 환경에 맞는 적절한 형태소 분석기를 사용해야 함

5.2.2 형태소 토큰화 라이브러리_NLTK

NLTK

- 자연어 처리를 위해 개발된 라이브러리
- 토큰화, 형태소 분석, 구문 분석, 개체명 인식, 감성 분석 등과 같은 기능 제공
- 주로 **영어 자연어 처리**를 위해 개발, 하지만 다양한 언어의 자연어 처리를 위한 데이터와 모델 제공
- 토큰화, 품사 태깅 작업을 위해 패키지나 모델을 다운로드 해야 함
 - Punkt 모델
 - 통계 기반 모델
 - Averaged Perception Tagger 모델
 - 퍼셉트론 기반 모델
 - 총 35개의 품사 태깅

5.2.3 형태소 토큰화 라이브러리_spacy

spaCy

- 사이썬을 기반으로 개발된 오픈 소스 라이브러리
- 자연어 처리를 위한 기능 제공
- **빠른 속도와 높은 정확도를 목표로 하는 머신 러닝 기반**의 자연어 처리 라이브러리 (NLTK 라이브러리와 차이!!)

NLTK와의 비교

- NLTK: **학습 목적**으로 자연어 처리에 대한 다양한 알고리즘 예제 제공
 - spaCy: **효율적인 처리 속도와 높은 정확도** 제공하는 것을 목표로 함
- ➔ spaCy에서 사용하는 모델이 더 크고 복잡하며 더 많은 리소스를 요구함

```
import spacy

nlp = spacy.load("en_core_web_sm")
sentence = "Those who can imagine anything, can create the impossible."
doc = nlp(sentence)

for token in doc:
    print(f"[{token.pos_:5} - {token.tag_:3}] : {token.text}")
```

5.3 하위 단어 토큰화



기존 형태소 분석기의 문제점

- OOV(Out-of-Vocabulary): 신규어/신조어/도메인 특화 용어 처리 취약
- 형태소 경계 의존: 띄어쓰기/품사 태깅 오류 전파
- 언어/도메인 편향: 특정 코퍼스 · 사전 설계에 과적합
- 유연성 부족: 다국어/도메인 확장 시 재설계/재학습 비용 큼

ex) “뇌-컴퓨터-인터페이스(BCI)” → 사전에 없으면 통째로 OOV

- 딥러닝 언어모델은 넓은 도메인, 빠른 신어 생성 속도에 대응 필요

하위 단어 토큰화

아이디어: 너무 미세(문자)도, 너무 거칠(단어)지도 않게 중간 단위로 분해

장점: OOV 감소, 어휘축소(메모리 · 연산 절감), 형태 변형(어미/접사) 대응

원칙: 자주 함께 나타나는 문자열을 묶어 서브워드 단위로 모델링

예시

"토크나이저" → "토크", "##나이", "##저"

- 원문: 시보리도 짱짱해고 허리도 어빙하지안구 죠하효
- 결과: ['시', '보리', '도', '짱짱해고', '허리', '도', '어', '빙하지안구', '죠', '하', '효']
- 원문: 진짜 멋있네요! 돈줄날만 하네요.
- 결과: ['진짜', '멋있네요', '!', '돈줄날', '만', '하네요', '.']

BPE(Byte Pair Encoding)

핵심 포인트

- 절차: (1) 문자 단위 시작 → (2) 가장 자주 출현하는 인접 쌍 병합 반복
- 산출물: 병합 규칙(merges) + 어휘(vocab)
- 특징: 빈도 기반, 간단 · 효율적, 신어 일반화 강함

예시①

원문 abracadabra

ab → A 치환: AracadAra → ra → B: ABcadAB → ca → C: CcadC

더 이상 병합할 쌍이 없으면 종료(토큰라이저에서는 병합 규칙을 축적)

예시 ②

코퍼스: "low lower lowest", "new newer", "widest"

초기: 문자 단위 토큰 [l][o][w] ...

반복 병합 예 (총 10회 가정)

1. 가장 빈번한 쌍 "l"+"o" → "lo" 병합
2. "lo"+"w" → "low"
3. "e"+"s" → "es" ...

결과

- 어휘 예: {"low", "er", "est", "new", ...}
- 임의 단어 분해: "lowest" → "low" + "est" → OOV 감소
- 한국어/영어 모두 적용 가능. 한국어는 공백 의존 줄이려면 ByteLevel/구두점 처리 주의

SentencePiece & Korpora(코포라) 라이브러리

SentencePiece

- 구글에서 개발한 오픈소스 하위 단어 토큰라이저 라이브러리
- 바이트 페어 인코딩과 유사한 알고리즘
- 사용자가 직접 설정할 수 있는 하이퍼파라미터 제공을 통해 세밀한 토큰나이징 기능 제공
- 언어 비의존: 공백 비의존, raw text 바로 학습(BPE/Unigram 알고리즘 지원)
- 장점: 데이터 정규화 · 특수 토큰 처리 포함, 재현성 우수

Korpora(코포라)

- 한국어 공개 코퍼스 수집 패키지(네이버 영화평 등) → 실습용 말뭉치 손쉽게 확보

pip install sentencepiece Korpora

표 5.5 SentencePieceTrainer 매개변수

매개변수	의미
input	말뭉치 텍스트 파일의 경로
model_prefix	모델 파일 이름
vocab_size	어휘 사전 크기
character_coverage	말뭉치 내에 존재하는 글자 중 토큰라이저가 다룰 수 있는 글자의 비율

model_type	토큰라이저 알고리즘
	■ unigram ■ bpe ■ char ■ word
max_sentence_length	최대 문장 길이
unk_id	어휘 사전에 없는 OOV를 의미하는 unk 토큰의 id (기본값: 0)
bos_id	문장이 시작되는 지점을 의미하는 bos 토큰의 id (기본값: 1)
eos_id	문장이 끝나는 지점을 의미하는 eos 토큰의 id (기본값: 2)

토큰나이저 학습 공통 프로세스

- 1.데이터 수집: 도메인 대표성/크기/클린
- 2.정규화: lowercasing, NFD/NFKC, 특수문자 규칙
- 3.프리토큰화: 공백/ByteLevel/Metaspace 등
- 4.모델 학습: BPE/WordPiece/Unigram, vocab 크기 결정
- 5.포스트프로세싱: [CLS]/[SEP] 삽입 등 템플릿 처리
- 6.평가: 토큰 길이, 커버리지, OOV, Perplexity 대리 지표
- 7.저장/배포: 버전 고정, 재현성/호환성 체크

실습① SentencePiece + Korpora(BPE)

1. 청원 말뭉치 로드 & 학습 파일 생성

목표: corpus.txt 생성

- load("korean_petitions") → 교재 말뭉치 사용.
- get_all_texts()로 한 줄 한 문장 형태 저장(학습 입력 준비).

```
from Korpora import Korpora

corpus = Korpora.load("korean_petitions")
petitions = corpus.get_all_texts()
# Create the 'datasets' directory if it doesn't exist
import os
os.makedirs("../datasets", exist_ok=True)
# Now open the file for writing
with open("../datasets/corpus.txt", "w", encoding="utf-8") as f:
    for petition in petitions:
        f.write(petition + "\n")
```

2. SentencePieceTrainer로 BPE 모델 학습

목표: petition_bpe.model, petition_bpe.vocab 생성

- model_type=bpe : 교재의 BPE 설정.
- vocab_size=8000 : 예시 값(교재 기준).
- 산출물 2개: .model(모델), .vocab(어휘).

```
from sentencepiece import SentencePieceTrainer
import os

# Create the 'models' directory if it doesn't exist
os.makedirs("../models", exist_ok=True)

SentencePieceTrainer.Train(
    "--input=../datasets/corpus.txt " # Added a space after .txt
    "--model_prefix=../models/petition_bpe " # Added a space after bpe
    "--vocab_size=8000 --model_type=bpe" # Added -- before model_type
)
```

실습① SentencePiece + Korpora(BPE)

3. SentencePieceProcessor로 encode/decode

목표: 토큰(pieces) · ID(ids) · 디코딩 확인

- encode_as_pieces/ids → 둘 다 교재 코드 동일.
- decode_*로 원복이 되는지 실습 확인.

```
from sentencepiece import SentencePieceProcessor

tokenizer = SentencePieceProcessor()
tokenizer.load("../models/petition_bpe.model")

sentence = "안녕하세요, 토큰라이저가 잘 학습되었군요!"
sentences = ["이렇게 입력값을 리스트로 받아서", "쉽게 토큰라이저를 사용할 수 있습니다"]

tokenized_sentence = tokenizer.encode_as_pieces(sentence)
tokenized_sentences = tokenizer.encode_as_pieces(sentences)
print("단일 문장 토큰화 :", tokenized_sentence)
print("여러 문장 토큰화 :", tokenized_sentences)

encoded_sentence = tokenizer.encode_as_ids(sentence)
encoded_sentences = tokenizer.encode_as_ids(sentences)
print("단일 문장 정수 인코딩 :", encoded_sentence)
print("여러 문장 정수 인코딩 :", encoded_sentences)

decode_ids = tokenizer.decode_ids(encoded_sentences)
decode_pieces = tokenizer.decode_pieces(encoded_sentences)
print("정수 인코딩에서 문장 변환 :", decode_ids)
print("하위 단어 토큰에서 문장 변환 :", decode_pieces)
```

```
단일 문장 토큰화 : ['_안녕하세요', ',', ' ', '_토', '크', '나', '이', '저', '가', '_잘', '_학', '습', '되', '었', '군', '요', '!']
여러 문장 토큰화 : [['_이렇게', '_입', '력', '값', '을', '_리', '스', '트', '로', '_받', '아', '서'], ['_쉽', '게', '_토', '크', '나', '이', '저', '를', '_사', '용', '할', '_수', '_있', '답', '니', '다']]
단일 문장 정수 인코딩 : [667, 6553, 994, 6880, 6544, 6513, 6590, 6523, 161, 110, 6554, 872, 787, 6648]
여러 문장 정수 인코딩 : [[372, 182, 6677, 4433, 1772, 1613, 6527, 4162], [1681, 994, 6880, 6544, 6513, 6590, 6536, 5852, 19, 5, 2639]]
정수 인코딩에서 문장 변환 : ['이렇게 입력값을 리스트로 받아서', '쉽게 토큰라이저를 사용할 수 있습니다']
하위 단어 토큰에서 문장 변환 : ['이렇게 입력값을 리스트로 받아서', '쉽게 토큰라이저를 사용할 수 있습니다']
```

실습① SentencePiece + Korpora(BPE)

4. SentencePiece 어휘 사전 확인 - vocab 조회 & 특수 토큰 확인

목표: <unk>, <s>, </s>와 vocab 크기 확인

- 교재 예시처럼 get_piece_size()로 전체 크기 확인.
- 특수 토큰 존재 확인(디코딩/마스킹에 중요).

```
from sentencepiece import SentencePieceProcessor

tokenizer = SentencePieceProcessor()
tokenizer.load("../models/petition_bpe.model")

vocab = {idx: tokenizer.id_to_piece(idx) for idx in range(tokenizer.get_piece_size())}
print(list(vocab.items())[:5])
print("vocab size :", len(vocab))
```

```
[(0, '<unk>'), (1, '<s>'), (2, '</s>'), (3, '니다'), (4, '___이')]
vocab size : 8000
```

WordPiece — 개요와 BPE와의 차이

- 목표: BERT 계열에서 채택. 확률기반 최대우도 기준으로 어휘 선정
- 차이점: BPE는 빈도 기반 병합, WordPiece는 언어모델 우도/점수를 사용
- 접두사: 보통 ##로 서브워드 표기(단어 중간)

점수식과 선택 원리

글자쌍 점수식 :
$$\text{score}(x, y) = \frac{f(x, y)}{f(x)f(y)}$$

f는 빈도, f(x,y)는 쌍 빈도 → 결합 강도가 큰 쌍을 우선 병합
BPE가 빈도 최다 쌍을 고른다면, WordPiece는 우도/점수 최대가 되는 쌍을 선택

교재 예시 흐름: 빈도 사전에서 점수 계산 → e+s, i+d 등 순차 병합
예시

문장: "자연어처리는 재미있다"

가능 서브워드 후보: "자연", "##어", "처리", "##는", "재미", "##있다" ...

선정 원리(직관)

- 문장 우도를 가장 늘리는 조합을 어휘에 포함
- 너무 희귀한 조각은 unk 처리 → 00V 최소화

발표 메모

- 구현체는 빈도/점수 혼합 휴리스틱. 핵심은 우도 기반 어휘 구성

Hugging Face tokenizers 라이브러리

특징

- 러스트 기반 고성능, 파이썬 바인딩
- BPE / WordPiece / Unigram / ByteLevel 등 지원
- 정규화/프리티즌화/포스트프로세싱 모듈화
- 불필요 공백 제거, 대소문자 변환, 유니코드 정규화, 구두점 처리, 특수 문자 처리 등을 제공
- 공백 혹은 구두점을 기준으로 입력 문장을 나눠 텍스트 데이터를 효과적으로 처리

왜 사용?

- 대용량 코퍼스에서도 빠른 학습
- 😊 transformers와 호환(저장 포맷/로더)

```
!pip install  
tokenizers
```

WordPiece 토큰나이저 모델 학습 과정

- 1.코퍼스 수집 · 정제(Korpora/사내 텍스트)
- 2.정규화 규칙 확정(NFKC, 숫자/기호 규칙)
- 3.프리토큰화(Whitespace/ByteLevel)
- 4.WordPiece 학습(어휘 크기, ## 접두사)
- 5.포스트프로세싱([CLS]/[SEP] 템플릿)
- 6.품질평가(토큰 길이/커버리지/OOV율)
- 7.버전관리 및 배포(JSON 저장)

주의

- 한국어는 공백 의존 낮음 → ByteLevel/Metaspace 고려

실습② tokenizers(WordPiece)

1.tokenizers로 WordPiece 구성 · 학습 · 저장

목표: petition_wordpiece.json 생성

- 표 5.6/5.7 중 실제 사용한 두 가지(NFD/Lowercase, Whitespace)만 채택.
- 저장 포맷은 JSON(모델+메타데이터+vocab 경로가 함께 기록).

표 5.6 정규화 클래스

이름	설명
NFD()	NFD 유니코드 정규화
NFKD()	NFKD 유니코드 정규화
NFC()	NFC 유니코드 정규화
...	...

표 5.7 사전 토큰화 클래스

클래스	설명
Sequence()	여러 개의 사전 토큰화 모듈을 병합하여 순서대로 수행
ByteLevel()	OpenAI에서 GPT-2를 학습할 때 사용한 방법으로 문자열을 그래픽 문자열로 매핑하고 공백을 기준으로 나눈다.
...	...

```
from tokenizers import Tokenizer
from tokenizers.models import WordPiece
from tokenizers.normalizers import Sequence, NFD, Lowercase
from tokenizers.pre_tokenizers import Whitespace
```

```
tokenizer = Tokenizer(WordPiece())
tokenizer.normalizer = Sequence([NFD(), Lowercase()])
tokenizer.pre_tokenizer = Whitespace()
```

```
# Update the path if corpus.txt is located in a different directory.
tokenizer.train("../datasets/corpus.txt")
tokenizer.save("../models/petition_wordpiece.json")
```

```
from tokenizers import Tokenizer
from tokenizers.decoders import WordPiece as WordPieceDecoder
```

```
tokenizer = Tokenizer.from_file("../models/petition_wordpiece.json")
tokenizer.decoder = WordPieceDecoder()
```

2. WordPiece 로드 & 디코더 설정

목표: 토큰화 결과를 자연스럽게 복원

- WordPiece는 ## 접두사 등 결합 규칙이 있어 디코더 설정 필수.

실습② tokenizers(WordPiece)

3. WordPiece 인코딩/배치 인코딩/디코딩 실습

목표: 토큰/ID/복원 결과 확인

- 교재와 동일 흐름: Encoding 객체 → .tokens/.ids 접근.
- decode()로 문장 복원 확인.

```
sentence = "안녕하세요, 토큰라이저가 잘 학습되었군요!"
sentences = ["이렇게 입력값을 리스트로 받아서", "쉽게 토큰라이저를 사용할 수 있습니다"]

encoded_sentence = tokenizer.encode(sentence)
encoded_sentences = tokenizer.encode_batch(sentences)

print("인코더 형식 :", type(encoded_sentence))

print("단일 문장 토큰화 :", encoded_sentence.tokens)
print("여러 문장 토큰화 :", [enc.tokens for enc in encoded_sentences])

print("단일 문장 정수 인코딩 :", encoded_sentence.ids)
print("여러 문장 정수 인코딩 :", [enc.ids for enc in encoded_sentences])

print("정수 인코딩에서 문장 변환 :", tokenizer.decode(encoded_sentence.ids))
```

정규화 · 프리토큰화 옵션 정리 (이번 실습)

- 정규화: NFD()(유니코드 분해), Lowercase()
- 사전 토큰화: Whitespace()
- (참고 항목) NFKC, Strip/StripAccents, Punctuation, Digits, ByteLevel, Metaspace ...
- 프로젝트 따라 교체 가능. 한국어는 공백 비의존 방법.
- ByteLevel/Metaspace도 유용하다.

06 임베딩



#6.0 임베딩 개요

컴퓨터는 텍스트 자체를 이해할 수 없기 때문에 텍스트 벡터화 과정이 필요함

텍스트 벡터화(Text Vectorization)

: 텍스트를 숫자로 변환하는 과정

원-핫 인코딩(One-Hot Encoding)

: 문서에 등장하는 각 단어를 고유한 색인 값으로 매핑 후 해당 색인 위치를 1로 표시, 나머지 위치는 모두 0으로 표시하는 방식

빈도 벡터화(Count Vectorization)

: 문서에서 단어의 빈도수를 세어 해당 단어의 빈도를 벡터로 표현하는 방식

장점: 단어나 문장을 벡터 형태로 변환하기 쉽다

단점: 벡터의 희소성(Sparsity)이 크다

컴퓨팅 비용의 증가, 차원의 저주

벡터가 텍스트의 의미를 반영하지 못한다

#6.0 임베딩 개요

워드 임베딩

: 단어를 고정된 길이의 실수 벡터로 표현하는 방법

- 단어의 의미를 벡터 공간에서 다른 단어와의 상대적 위치로 표현해 단어 간의 관계를 추론

대표 기법

Word2Vec, fastText

단점

고정된 임베딩을 학습하기 때문에 다의어나 문맥 정보를 다루기 어렵다

-> 동적 임베딩(Dynamic Embedding) 기법 사용

#6.1 언어 모델

언어모델(Language Model)

: 입력된 문장으로 각 문장을 생성할 수 있는 확률을 계산하는 모델

- 주어진 문장을 바탕으로 문맥을 이해하고, 문장 구성에 대한 예측을 수행

활용 분야: 자연어 처리 분야 (자동 번역, 음성 인식, 텍스트 요약 등)

언어 모델 구조

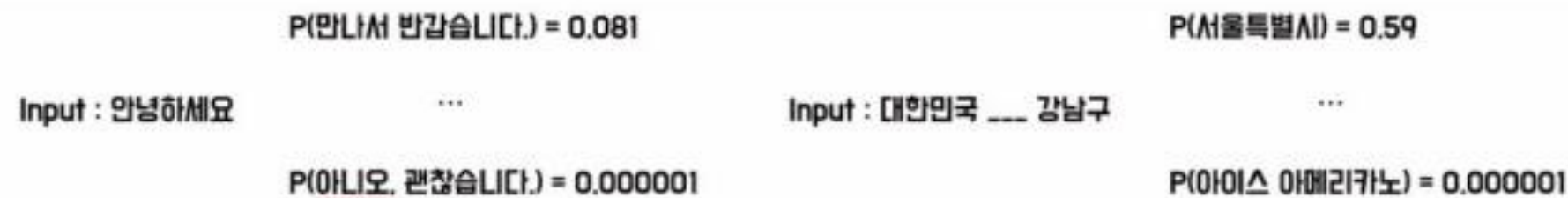


그림 6.1 언어 모델

단점

주어진 문장 뒤에 나올 수 있는 문장이 다양하기 때문에 완성된 문장 단위로 확률을 계산하는 것은 어려움

-> 자기회귀 언어 모델: 하나의 토큰 단위로 예측

#6.1 언어 모델

자기회귀 언어 모델(Autoregressive Language Model)

- 입력된 문장들의 조건부 확률을 이용해 다음에 올 단어를 예측

조건부 확률: 이전 단어들의 시퀀스가 주어졌을 때, 다음 단어의 확률을 예측하는 것

$$P(w_t | w_1, w_2, \dots, w_{t-1}) = \frac{P(w_1, w_2, \dots, w_t)}{P(w_1, w_2, \dots, w_{t-1})}$$

➔ 여기에 조건부 확률의 연쇄법칙(Chain rule for conditional probability) 적용

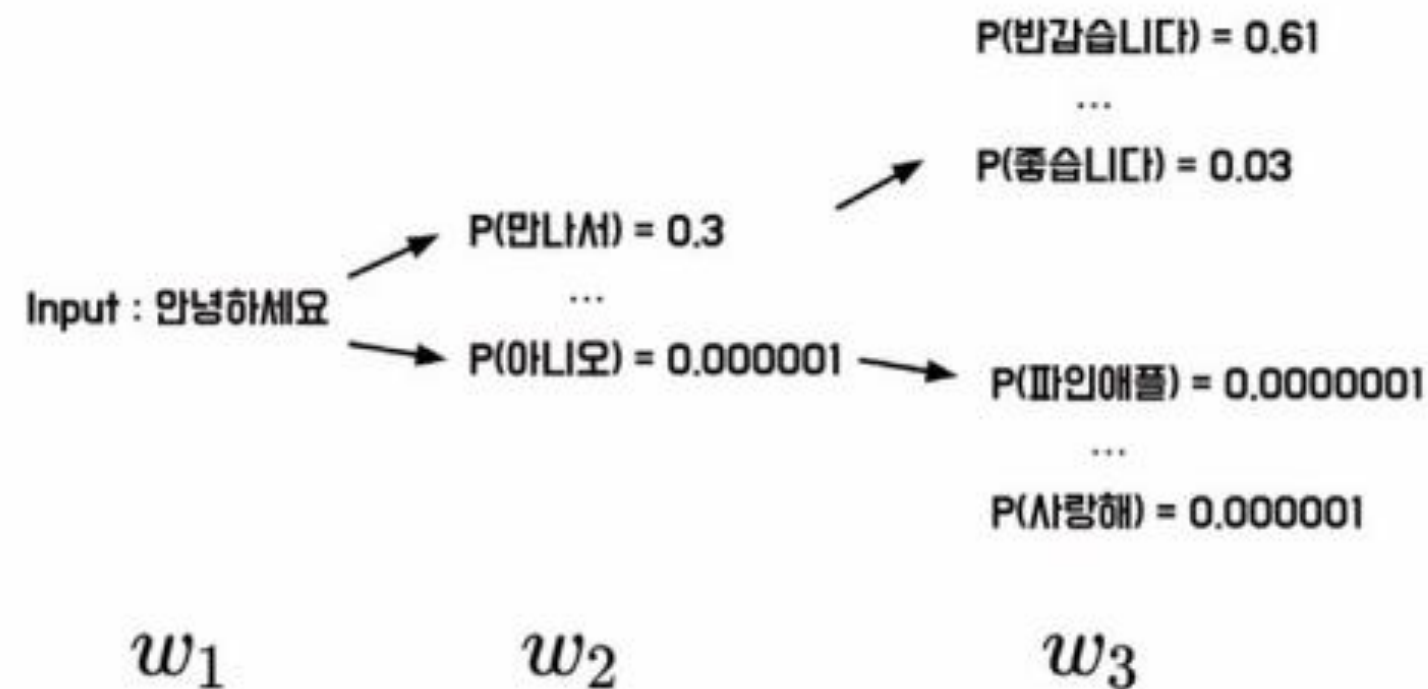
$$P(w_t | w_1, w_2, \dots, w_{t-1}) = P(w_1)P(w_2 | w_1) \cdots P(w_t | w_1, w_2, \dots, w_{t-1})$$

➔ 이를 통해 언어 모델은 문장 전체의 확률을 계산하고, 그것을 이용해 다음 단어를 예측

#6.1 언어 모델

자기회귀 언어 모델(Autoregressive Language Model)

ex) '안녕하세요' 다음에 등장할 수 있는 문장을 조건부 확률의 연쇄법칙 방법으로 시각화



- 자기회귀 언어 모델에서는 각 시점에서 다음에 올 토큰을 예측하는 것이 중요
-> 토큰 분류 문제로 정의할 수 있음
- 모델의 출력값이 모델의 입력값으로 사용됨
- 문장 전체의 확률을 계산하고, 이를 이용해 다음 단어를 예측

#6.1 언어 모델

통계적 언어 모델 (Statistical Language Model)

- 언어의 통계적 구조를 이용해 문장이나 단어의 시퀀스를 생성하거나 분석
- 시퀀스에 대한 확률 분포를 추정 -> 문장의 문맥을 파악해 다음에 등장할 단어의 확률 예측
- 마르코프 체인(Markov Chain)을 이용해 구현됨

마르코프 체인

: 빈도 기반의 조건부 확률 모델로, 이전 상태와 현재 상태 간의 전이 확률을 이용해 다음 상태 예측

빈도 기반의 조건부 확률

$$P(\text{만나서} | \text{안녕하세요}) = \frac{P(\text{안녕하세요 만나서})}{P(\text{안녕하세요})} = \frac{700}{1000}$$

$$P(\text{반갑습니다} | \text{안녕하세요}) = \frac{P(\text{안녕하세요 반갑습니다})}{P(\text{안녕하세요})} = \frac{100}{1000}$$

→ 데이터 희소성(Data sparsity) 문제 발생

#6.1 언어 모델

GPT와 BERT의 문장 생성 기법

GPT

- Raw Sentence: GPT는 OpenAI에서 개발한 인과적 언어 모델입니다.
- Input: GPT는 OpenAI에서 개발한
- Prediction-1: 인과적
- Prediction-2: 언어 모델입니다.

BERT

- Raw Sentence: BERT는 Google에서 개발한 마스킹된 언어 모델입니다.
- Input: Bert는 [MASK]에서 개발한 [MASK] 언어 모델입니다.
- Prediction: Google, 마스킹된

- 통계적 언어 모델은 기존에 학습한 텍스트 데이터에서 패턴을 찾아 확률 분포 생성
 - > 이를 이용해 새로운 문장 생성/다양한 종류의 텍스트 데이터 학습 가능
- 대규모 자연어 데이터 처리하는 데 효과적

#6.2 N-gram

N-gram

- 가장 기초적인 통계적 언어 모델
- 텍스트에서 N개의 연속된 단어 시퀀스를 하나의 단어로 취급-> 특정 단어 시퀀스가 등장할 확률 추정
- 유니그램(N=1), 바이그램(N=2), 트라이그램(N=3), N-gram(N>=4)

입력 문장을 분할한 예시

Unigram : 안녕하세요 만나서 진심으로 반가워요

안녕하세요. 만나서. 진심으로. 반가워요

Bigram : 안녕하세요 만나서 진심으로 반가워요

안녕하세요 만나서. 만나서 진심으로. 진심으로 반가워요

Trigram : 안녕하세요 만나서 진심으로 반가워요

안녕하세요 만나서 진심으로. 만나서 진심으로 반가워요

t번째 토큰의 조건부 확률을 계산하는 수식

$$P(w_t | w_{t-1}, w_{t-2}, \dots, w_{t-N+1})$$

- 이전 단어들의 개수를 결정하는 N의 값을 조정하여 N-gram 모델의 성능을 조절

#6.2 N-gram

N-gram 구현 방법

```
import nltk
def ngrams(sentence, n):
    words = sentence.split()
    ngrams = zip(*[words[i:] for i in range(n)])
    return list(ngrams)

sentence = "안녕하세요 만나서 진심으로 반가워요"

unigram = ngrams(sentence, 1)
bigram = ngrams(sentence, 2)
trigram = ngrams(sentence, 3)
```

```
print(unigram)
print(bigram)
print(trigram)

unigram = nltk.ngrams(sentence.split(), 1)
bigram = nltk.ngrams (sentence.split(), 2)
trigram = nltk.ngrams (sentence.split(), 3)

print(list(unigram))
print(list(bigram))
print(list(trigram))
```

출력 결과

```
[('안녕하세요',), ('만나서',), ('진심으로',), ('반가워요',)]
[('안녕하세요', '만나서'), ('만나서', '진심으로'), ('진심으로', '반가워요')]
[('안녕하세요', '만나서', '진심으로'), ('만나서', '진심으로', '반가워요')]
[('안녕하세요',), ('만나서',), ('진심으로',), ('반가워요',)]
[('안녕하세요', '만나서'), ('만나서', '진심으로'), ('진심으로', '반가워요')]
[('안녕하세요', '만나서', '진심으로'), ('만나서', '진심으로', '반가워요')]
```

활용 분야

관용적 표현 분석, 작은 규모의 데이터셋에서 연속된 문자열 패턴 분석, 자연어 처리 작업 및 문자열 패턴 분석

#6.3 TF-IDF

TF-IDF(Term Frequency-Inverse Document Frequency)

- : 텍스트 문서에서 특정 단어의 중요도를 계산하는 방법
- 문서 내에서 단어의 중요도를 평가하는 데 사용되는 통계적 가중치
- Bow에 가중치를 부여하는 방법

BoW (Bag-of-Words)

- : 문서나 문장을 단어의 집합으로 표현하는 방법
- 문서/문장에 등장하는 단어의 중복을 허용해 빈도를 기록

Bow 벡터화 예시

- 모든 단어는 동일한 가중치를 갖는다

	I	like	this	movie	don't	famous	is
This movie is famous movie	0	0	1	2	0	1	1
I like this movie	1	1	1	1	0	0	0
I don't like this movie	1	1	1	1	1	0	0

#6.3 TF-IDF

단어 빈도 (Term Frequency, TF)

: 문서 내에서 특정 단어의 빈도수를 나타내는 값

- 해당 단어의 상대적인 중요도를 측정하는 데 사용
- 높은 TF -> 특정 문서에서 중요한 역할 / 전문 용어나 관용어

$$TF(t, d) = count(t, d)$$

문서 빈도 (Document Frequency, DF)

: 한 단어가 얼마나 많은 문서에 나타나는지

- 높은 DF -> 일반적으로 널리 사용되는 단어 / 중요도 낮음
- 낮은 DF -> 특정 문맥에서만 사용되는 단어 / 중요도 높음

$$DF(t, D) = count(t \in d : d \in D)$$

역문서 빈도(Inverse DF, IDF)

: 문서 내에서 특정 단어가 얼마나 중요한지

- 전체 문서 수를 문서 빈도로 나눈 다음 로그 취한 값
- 높은 IDF -> 일반적이고 중요하지 않은 단어

$$IDF(t, D) = \log\left(\frac{count(D)}{1 + DF(t, D)}\right)$$

TF-IDF

: 문서 빈도와 역문서 빈도를 곱한 값

- 관사/관용어 등의 가중치 낮아짐

$$TF-IDF(t, d, D) = TF(t, d) \times IDF(t, d)$$

#6.3 TF-IDF

TF-IDF 클래스의 주요 매개변수

- 입력값(input): 입력될 데이터의 형태
 - content: 문자열 데이터/바이트 형태의 입력 값
 - file: 파일 객체 사용
 - filename: 파일 경로 사용
- 인코딩(encoding): 바이트/파일을 입력값으로 받을 경우 사용할 텍스트 인코딩 값
- 소문자 변환(lowercase): 입력받은 데이터를 소문자로 변환할지 여부
 - True: 모든 입력 데이터를 소문자로 변환
- 불용어(stop_words): 분석에 도움이 되지 않는 의미 없는 단어
 - 단어 사전에 추가되지 않음
- N-gram 범위(ngram_range): 사용할 N-gram의 범위
 - (최솟값, 최댓값) 형태로 입력
 - (1, 1): 유니그램
 - (1, 2): 유니그램과 바이그램을 사용
- 최댓값 문서 빈도(max_df): 전체 문서 중 일정 횟수 이상 등장하는 단어는 불용어 처리
 - 정수 입력: 해당 횟수 초과해 등장하는 단어를 불용어 처리
 - 1이하 실수 입력: 해당 비율을 초과해 등장한 단어를 불용어 처리
- 최솟값 문서 빈도(min_df): 전체 문서 중 일정 횟수 미만으로 등장한 단어를 불용어 처리
- 단어사전(vocabulary): 미리 구축한 단어 사전
- IDF 분모처리(smooth_idf): IDF 계산 시 분모에 1 더함

#6.3 TF-IDF

TF-IDF 클래스 활용해 벡터화 진행

```
from sklearn.feature_extraction.text import TfidfVectorizer

corpus = [
    "That movie is famous movie",
    "I like that actor",
    "I don't like that actor"
]

tfidf_vectorizer = TfidfVectorizer()
tfidf_vectorizer.fit(corpus)
tfidf_matrix = tfidf_vectorizer.transform(corpus)

print(tfidf_matrix.toarray())
print(tfidf_vectorizer.vocabulary_)
```

출력 결과

```
[[0.          0.          0.39687454 0.39687454 0.          0.79374908 0.2344005 ]
 [0.61980538 0.          0.          0.          0.61980538 0.          0.48133417]
 [0.4804584  0.63174505 0.          0.          0.4804584  0.          0.37311881]]
{'that': 6, 'movie': 5, 'is': 3, 'famous': 2, 'like': 4, 'actor': 0, 'don': 1}
```

TF-IDF 객체 생성 및 학습 과정

1. 객체 불러오기
 - TfidfVectorizer 클래스로 tfidf_vectorizer 객체 생성
2. fit() 메서드
 - fit() 메서드를 통해 말뭉치 학습
 - 모델이 단어 사전을 구축
3. transform() 메서드
 - 학습된 객체를 사용해 데이터를 TF-IDF 벡터로 변환
4. fit_transform() 메서드
 - 학습과 변환을 한 번에 수행

#6.3 TF-IDF

주요 메서드 및 속성

- `toarray()`: TF-IDF 결과를 넘파이 배열 형태로 변환
- 배열 형태: (문서 수, 단어 수)의 2차원 형태
- `vocabulary_` 속성
 - 키(key): 단어
 - 값(value): 단어의 색인(index)

TF-IDF의 한계점

- 문맥과 단어의 순서를 고려하지 않음
 - > 문장 생성이나 문장 순서가 중요한 작업에 부적합
- 단어의 의미를 반영하지 않음
 - > TF-IDF 벡터는 중요도만 나타내며, 단어 간 의미적 유사성을 표현하지 못함

THANK YOU

